Routing Configuration and Wireframes

1 Routing in a single-page application

In the first phase of the thesis, I will use React (a UI library) to set up App shells, and the React Router package will be used for routing.
In a single-page application (SPA), navigating to a new URL does not request a new HTML document from the server. Instead, the History API method `history.pushState()` or `history.replaceState()` adds a new entry to the browser's session history and updates the URL in the address bar without a full page reload, and JavaScript then renders the content for the new URL on the client. During back and forward navigation, the browser dispatches the `popstate` DOM event. Every listener registered for this event is notified that the URL has changed, reads the current location from `window.location`, and re-renders the content accordingly.
React Router abstracts this process: `<Link>`, `<Navigate>` and `useNavigate()` call `history.pushState()` or `history.replaceState()` during navigation. A router component (React Router offers several; this project uses `BrowserRouter`) subscribes to the browser history, keeps track of the current location and provides it through the router context. `<Routes>` reads the current location from this context, matches it against its child `<Route>` definitions and renders the element of the best-matching route in the DOM tree.
In an SPA, typically one app instance is running and a single router context exists. Calling `history.pushState()` or `history.replaceState()` does not dispatch any event, so only the code that called it knows that the URL has changed: in React Router, this is the router whose context the navigating component belongs to. In a single app this is not a problem, because that router also updates its own location and re-renders the UI. During back and forward navigation, in contrast, the browser dispatches a `popstate` event, so every listener on the page is notified of the URL change.
In a micro frontend setup, more than one application instance (each with its own React root and router) can be mounted at the same time. How, then, do these routers get to know about a change of the browser URL? When a navigation happens in one app, only the router of that app knows about the URL change. The routers of the other app instances are not notified and keep rendering the UI for the old URL until a back or forward navigation fires a real `popstate` event.
An app shell built with a micro frontend framework such as single-spa or Piral solves this routing problem. In the future, I will build a custom React shell, so addressing the routing problem early and building a mental model of it will be helpful for later development. The options described in this document are not final and may change in the future.

The application to be built is a business-to-consumer (B2C) online clothing shop. It consists of an app shell and three MFEs: product-mfe, cart-mfe and user-mfe. Each MFE has its own unique URL prefix:

MFE	URL prefix
product-mfe	/30722
cart-mfe	/30723
user-mfe	/30724

so not sth like
/product
/cart
/user

but rather a numeric prefix
totally fine if you desire it like this
to technically separate (like different ports)

and it is not assumed
that user type in bookmark links
but rather navigate through the app

please consider that sth like
a route should be
stable as long as possible
to allow bookmarks

For routing, I have two options in mind.

2 Option 1: The shell owns the only router on the page

There is one `BrowserRouter` on the page and that is shell's BrowserRouter, so every navigation, from any MFE, reaches the same router provider. The shell's route config maps each URL to a layout and the MFEs that fill it. The shell matches the URL against this config to decide what to render. MFEs are loaded at runtime and rendered as ordinary React components inside the shell's tree.

1. Each MFE exposes its pages as `<Routes>` (or route objects) without its own router. Its routes read the location from the shell's router context.
2. `<Link>`, `<Navigate>` and `useNavigate()` in any MFE call `history.pushState()` and then notify the shell's router directly. Back/forward fires the browser's `popstate` event, which the same router listens to. In both cases, the shell re-renders, re-evaluates the layout, and every mounted MFE re-matches its `<Routes>`.

The following table shows the router config for this option. If the layout is a horizontal split, both the top and the bottom column are filled. The bottom area can hold more than one MFE If the layout is full height, only the top column is filled, and the bottom column is empty (–).

URL	Top MFE and page	Bottom MFE(s) and page	Wireframe
/ / === /30722 default I guess	product-mfe LandingPage	–	Fig.1
`/30722/fav`	product-mfe FavoritesPage	–	Fig. 3
`/30724/user`	user-mfe UserPage	–	Fig. 4 (left)
`/30724/login`	user-mfe LoginPage	–	Fig. 4 (right)
`/30724/register`	user-mfe RegisterPage	–	Fig. 5
`/30724/login/verify`	user-mfe VerifyLoginPage	–	–
`/30722/csc/:categoryId/:subcategoryId?`	product-mfe ProductsPage	–	Fig. 7
`/30722/p/:productId`	product-mfe ProductDetail	–	Fig. 8
`/30723/cart`	cart-mfe CartPage	product-mfe RecommendationAtCart	Fig. 9
`/30723/checkout`	cart-mfe CheckoutPage	product-mfe RecommendationAtCart	Fig. 10 (left)
`/30723/checkout/login`	cart-mfe CheckoutAskToLoginPage	user-mfe LoginPage product-mfe RecommendationAtCartPage	Fig. 10 (right)
`/30723/checkout/register`	cart-mfe CheckoutAskToRegisterPage	user-mfe RegisterPage product-mfe RecommendationAtCartPage	Fig. 11

where is the config coming from that the shells knows to mount product or user mfe?
or is product MFE always present (default) and only on selected pages, user-mfe added on top

are there 1 or 2 named router outlets (HTML projection slot placeholder) at the bottom so if product is present it always appears in same html node? or do you have two numbered slots and product takes „2“ and if user is not present, „1“ is empty and collapses?
or is the shell adding new entry nodes for those upon config?

this is the added complexity of a horizontal vs a plain vertical split, yes

Drawback: The shell's route config must know every URL and page of every MFE. When an MFE adds a new page or changes one of its URLs, the shell's config has to be updated and the shell redeployed as well, so an MFE can no longer be released independently of the shell. In addition, an MFE's routes can only read the shell's router context if the shell and all MFEs share the same instance of React and React Router at runtime. If an MFE bundles its own copy, its `<Routes>` and `useNavigate()` do not find the shell's router and fail. Every MFE is therefore tied to the shell's versions of these libraries, and no MFE can upgrade React Router on its own.

3 Option 2: First-level routing in the shell and second-level routing in each MFE

The shell's router decides which MFEs are loaded and mounted. Each mounted MFE runs its own `BrowserRouter` for its internal pages.
Because `pushState()` does not dispatch any event, the routers would not know about each other's navigations. To solve this, the shell replaces `history.pushState()` and `history.replaceState()` with its own versions at startup. On every URL change (every time one of these methods is called, by the shell or by any MFE), the shell's version first calls the original method and then dispatches a synthetic `popstate` DOM event. All routers on the page hear this event and sync with the URL change. Back and forward need no help: the browser fires a real `popstate` event, which all routers already listen to.
In this option, each MFE's router uses the MFE's URL prefix as its basename.

- *First-level routing (shell):* The shell mounts an MFE based on the basename at the start of the URL. For example, when the URL is `/30723/cart`, the shell mounts cart-mfe because the URL starts with `/30723`.
- *Second-level routing (MFE):* Inside the mounted MFE, its own router matches the rest of the URL. cart-mfe has the basename `/30723`, so it matches `/cart` and renders the cart page.
- *Horizontal splitting (special cases):* Some URLs are defined as special cases where a second MFE is mounted below the first one. A **split rule** in the shell's configuration defines such a case. It consists of a URL pattern, the additional MFE to mount, and the position of each MFE. For example, when the URL path is `/30723/cart`, the shell mounts cart-mfe (top) because of the basename, and it also mounts product-mfe (bottom) because of a split rule. product-mfe has its own route and layout for the full path `/30723/cart`, so it knows what to render in this case.

Advantage: Each MFE has its own React root and its own copy of React Router, so it can use any React Router version, because every router is notified of a URL change through the global `popstate` event instead of a shared router context.
Drawback: The split rules are defined in two places: in the shell, which decides to mount the additional MFE, and in that MFE's own routes, which decide what it renders. If a URL is changed in one place without updating the other, part of the page breaks: the slot either stays empty or renders the wrong content. In addition, the bottom MFE must know URLs that belong to top MFE(product-mfe knows `/30723/cart`, which belongs to cart-mfe), which couples the two MFEs.

sounds complex

also conflicts of router state expected

do you used named router outlets / same slots for product main vs product bottom e.g.

would be nice to see the HTML slot layout and the actual router config code

4 Wireframes walkthrough

This section walks through the wireframes of the application. Each wireframe is shown with its URL, and annotated bounding boxes mark every MFE that is loaded at that time.

4.1 Landing page view

URL: /
When the user first loads the application, whether as a guest or as a logged-in user, the landing page view shown in Figure 1 is displayed. Rendering the landing page does not depend on the authentication state. At this point, product-mfe is active along with the app shell. The app shell owns the header at the top and the footer at the bottom, while product-mfe owns the content area in between.
When the user hovers over one of the items in the navigation bar, a drawer opens beneath it, as shown in Figure 2. The drawer displays the subcategories of that navigation item, giving the user a quick overview of the available choices.

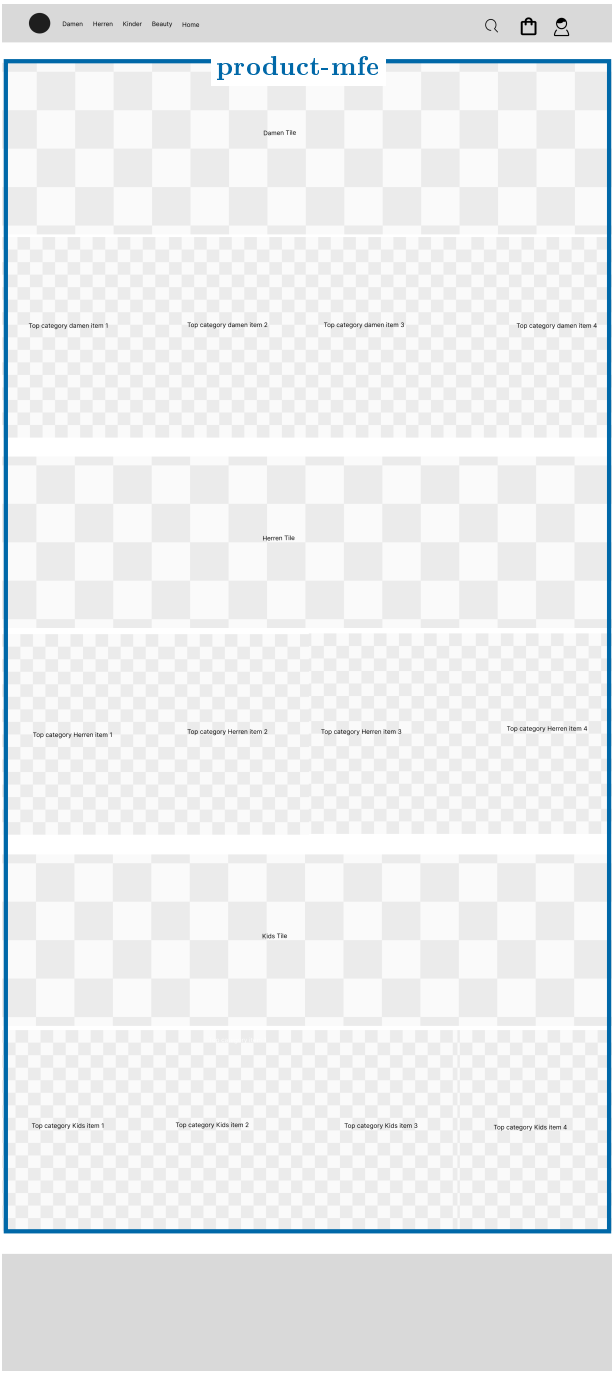


Figure 1: Landing page

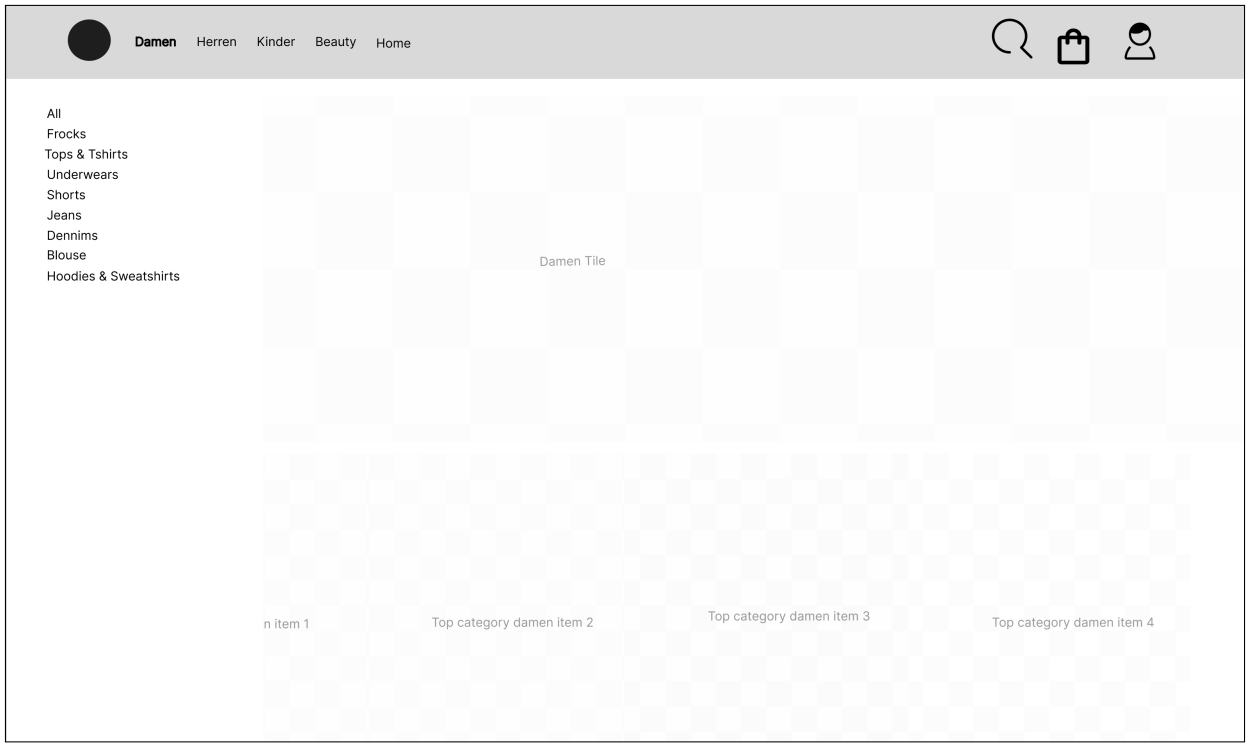


Figure 2: Navigation drawer opened below the hovered navigation item

4.2 Favourite view

URL: `/30722/fav`
When the user clicks on the heart icon in the header, the favourite view shown in Figure 3 is rendered. It lists all products that the user has marked as favourites by clicking the heart icon on a product card. As on the landing page, product-mfe is active along with the app shell.

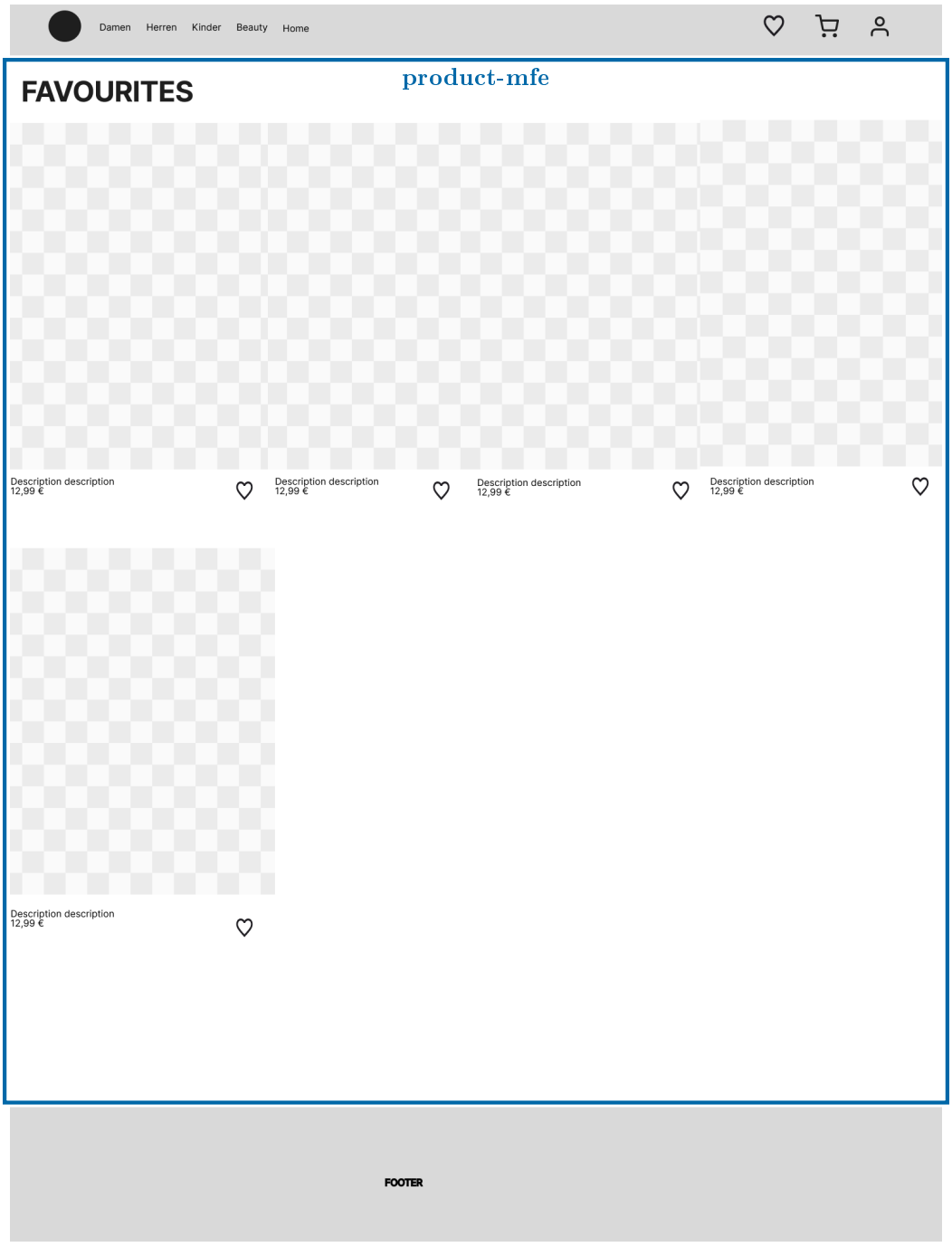
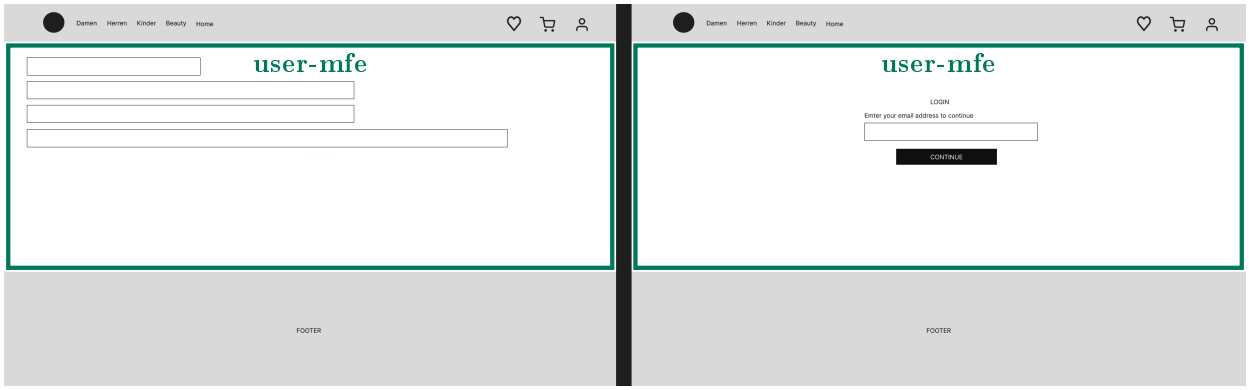


Figure 3: Favourite view

4.3 User view

URL: `/30724/user` (logged-in user), `/30724/login` (guest user)
When the user clicks on the user icon in the header, the user view shown in Figure 4 is rendered. The figure shows both variants of this view side by side: the left side corresponds to a logged-in user, and the right side to a guest user. A logged-in user sees their user details together with their order details. A guest user is redirected from `/30724/user` route to `/30724/login` route and sees a login form that asks for their email address. In both viewes, user-mfe is active along with the app shell.



Logged-in user Guest user

Figure 4: User view for a logged-in user (left) and a guest user (right)

4.3.1 Register view

URL: `/30724/register`
For a guest user, if the email address entered in the login form does not belong to a registered user, the user is redirected to the register view shown in Figure 5. Here, the user can create an account by entering an email address and a password and then repeating the password. As in the user view, user-mfe is active along with the app shell.

4.3.2 Login confirmation view

URL: `/30724/login` or `/30724/register` (unchanged)
Once the user has successfully submitted the login form or the register form, a magic login link is sent to the user's email address, and the confirmation view shown in Figure 6 is rendered. It asks the user to check their inbox and to click the link to log in. Again, user-mfe is active along with the app shell.
When the user clicks on the link, they are first taken to `/30724/login/verify`, where the login is verified. Then they are redirected to the landing page at `/` as logged-in users and the authentication token is stored in a cookie with an expiration period of 30 days.

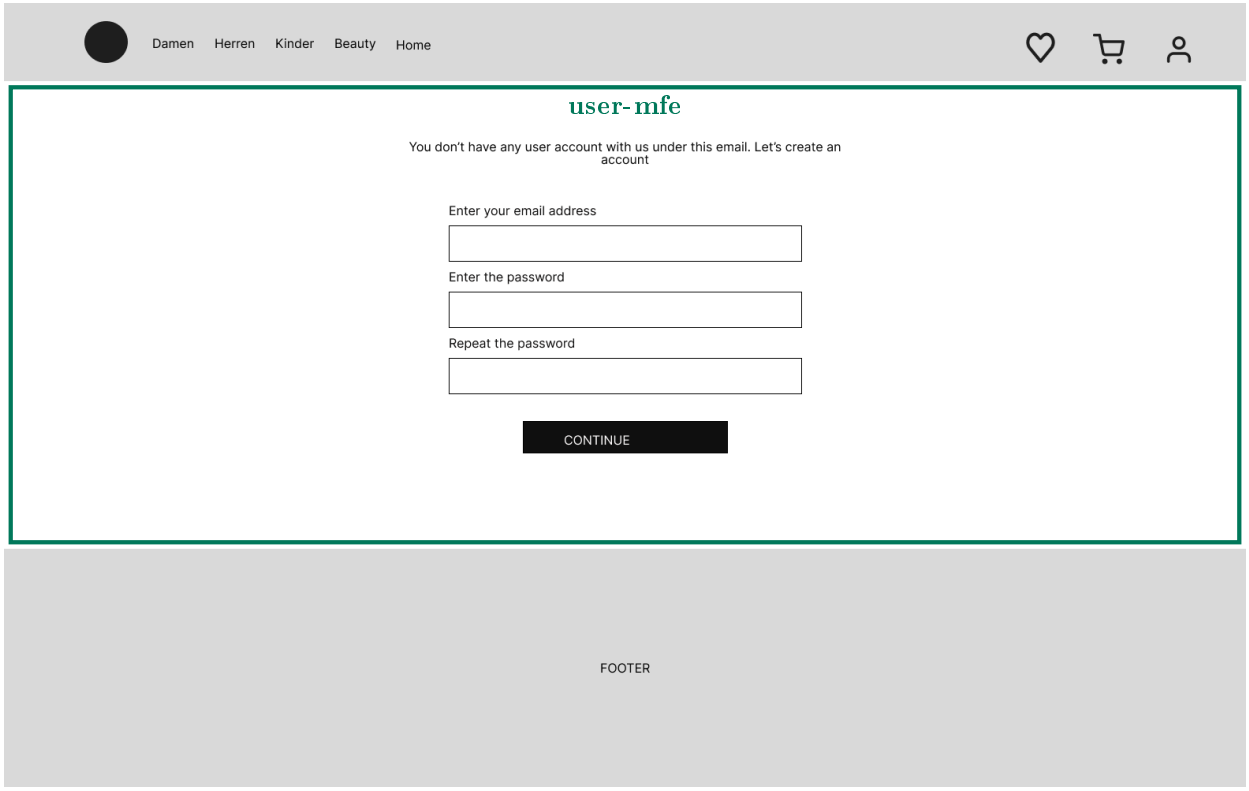


Figure 5: Register view

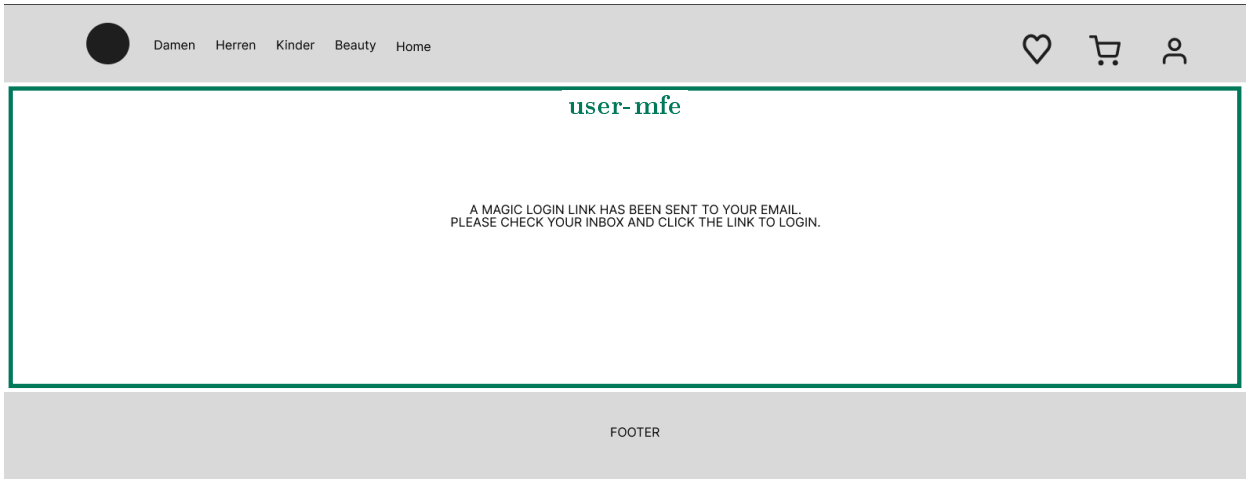


Figure 6: Login confirmation view after a successful login or registration

4.4 Products View

URL: `/30722/csc/:categoryId/:subcategoryId?`
When the user clicks on a subcategory on a drawer, Figure 2, whether as a guest or as a logged-in user, the view shown in Figure 7 is rendered. Rendering the products page view does not depend on the authentication state. As on the landing page, product-mfe is active along with the app shell.

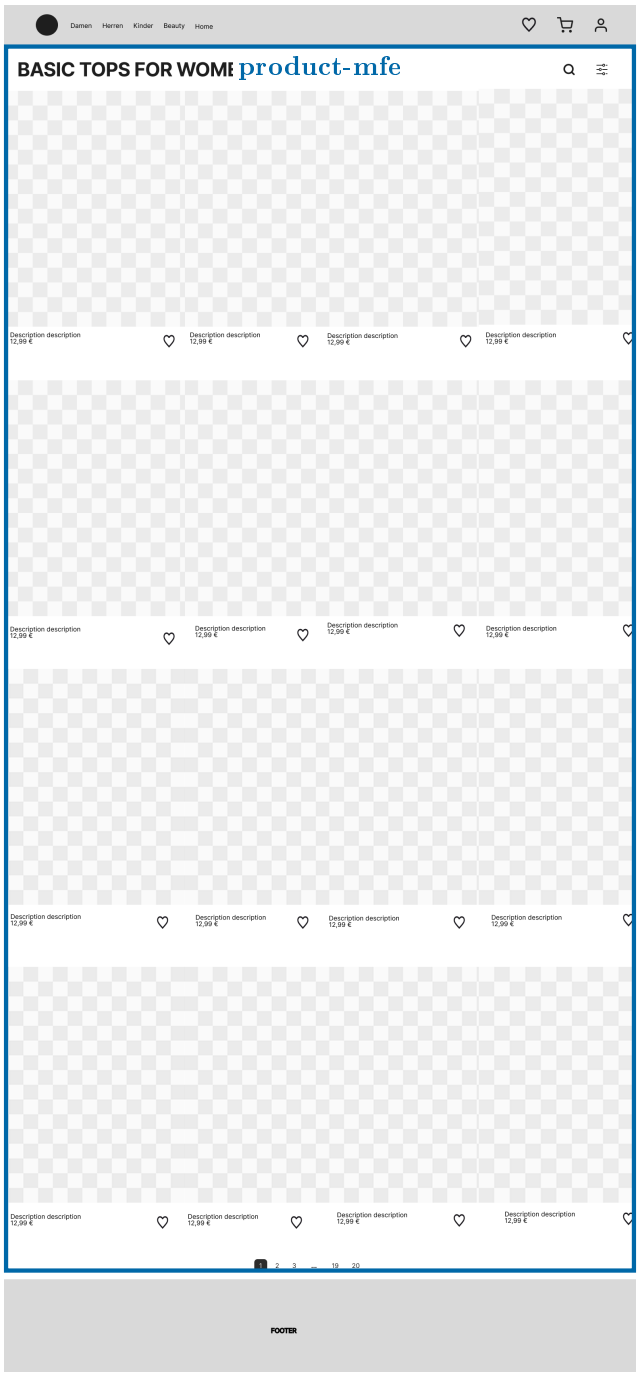
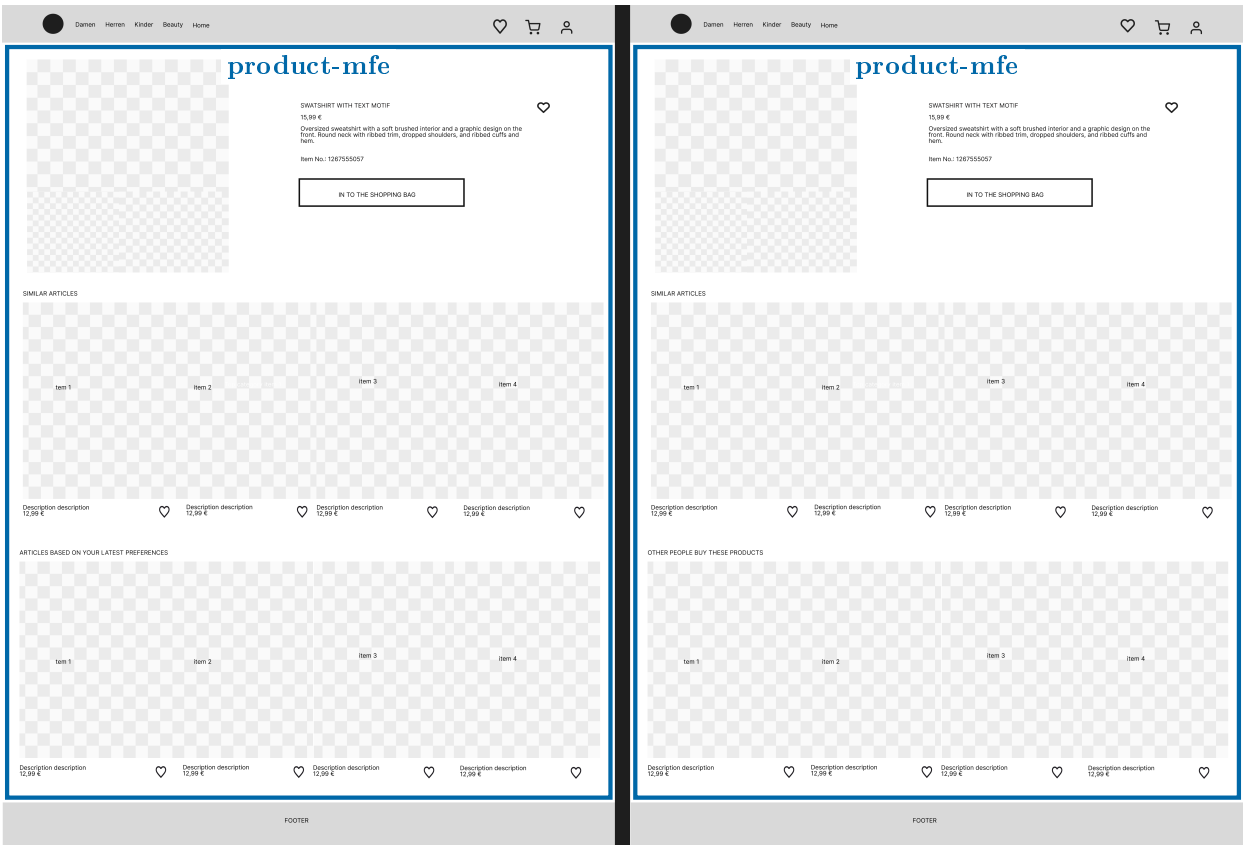


Figure 7: Products page

The products page also supports searching, filtering and pagination. The current search query, the applied filter parameters and the current page number are all reflected as query parameters in the page URL. As a result, the exact state of the product listing, including the search term, filters and page, can be bookmarked or shared, and revisiting that URL later reproduces the same view.

4.5 Product details view

URL: `/30722/p/:productId`
When the user clicks on a product card, the product details page shown in Figure 8 is rendered. The figure shows both variants of this page side by side: the left side corresponds to a logged-in user, and the right side to a guest user. The two variants differ only in the last widget: a logged-in user sees articles based on their latest preferences, whereas a guest user sees products that other people bought. As in the previous views, product-mfe is active along with the app shell.

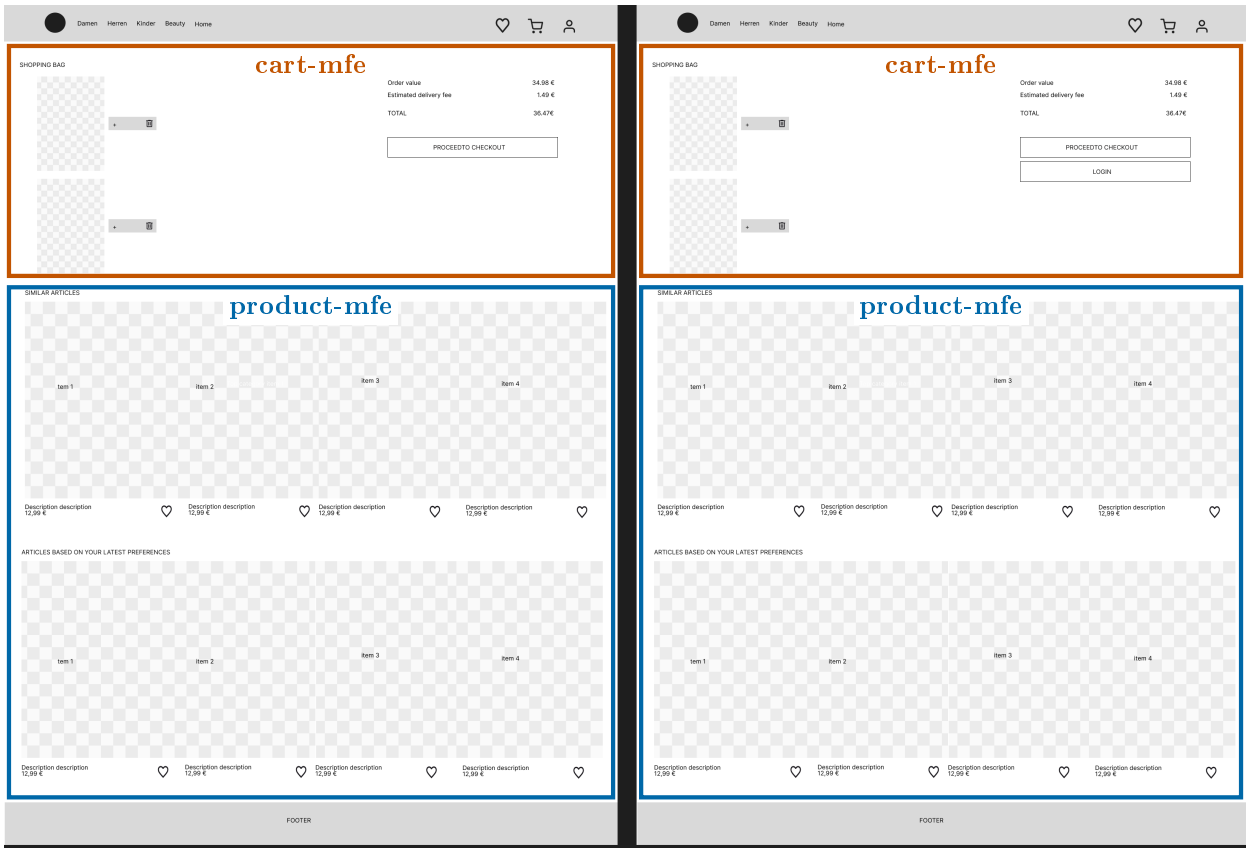


Logged-in user Guest user

Figure 8: Product details view for a logged-in user (left) and a guest user (right)

4.6 Cart view

URL: `/30723/cart`
When the user clicks on the shopping bag icon in the header, the shopping cart view shown in Figure 9 is rendered. Unlike the views discussed so far, this view follows the horizontal split pattern: cart-mfe occupies the top section directly below the header, and product-mfe occupies the bottom section above the footer. The figure shows both variants of this page side by side: the left side corresponds to a logged-in user, and the right side to a guest user. In the guest variant, a login button is shown below the proceed-to-checkout button, because a guest user has to log in during the checkout step anyway. In addition, the second widget of product-mfe differs slightly for a guest user.

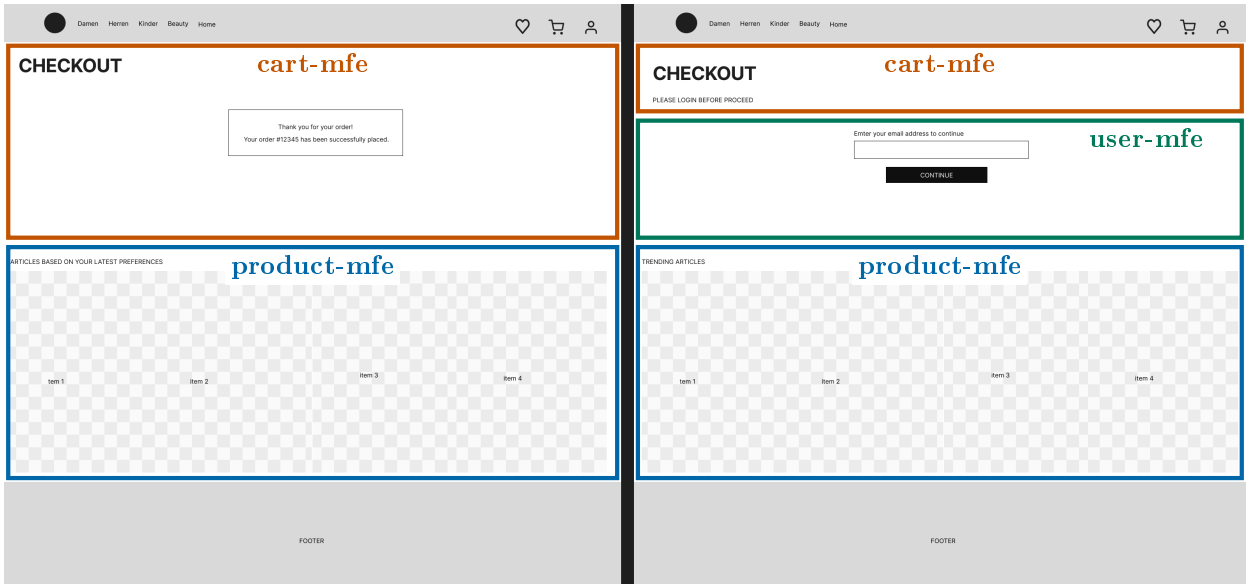


Logged-in user Guest user

Figure 9: Cart view for a logged-in user (left) and a guest user (right)

4.7 Checkout view

URL: `/30723/checkout` (logged-in user), `/30723/checkout/login` (guest user)
When the user clicks on the proceed to checkout button in the cart view, the checkout view shown in Figure 10 is rendered. The figure shows both variants of this view side by side: the left side corresponds to a logged-in user, and the right side to a guest user. For a logged-in user, the order is placed, and a success message with the order number is displayed. A guest user, in contrast, is redirected to the checkout login route, `/30723/checkout/login` and is asked to log in before proceeding.
For a logged-in user, cart-mfe occupies the top section and product-mfe the bottom section, as in the cart view. For a guest user, three MFEs are active on the page for the first time: cart-mfe occupies the top section, and the bottom part is shared by user-mfe, which renders the login component in its upper half, and product-mfe, which renders trending articles in its lower half.



Logged-in user Guest user

Figure 10: Checkout view for a logged-in user (left) and a guest user (right)

4.7.1 Checkout register view

URL: `/30723/checkout/register`
If the email address entered in the checkout login form does not belong to a registered user, the view shown in Figure 11 is rendered. Instead of its login component, user-mfe now loads its register component, where the user can create an account. All three MFEs are still active: cart-mfe occupies the top section, and the bottom part is again shared by user-mfe in its upper half and product-mfe in its lower half.

4.7.2 Checkout login confirmation view

URL: `/30723/checkout/login` or `/30723/checkout/register` (unchanged)
Once the user has successfully submitted the login form or the register form during checkout, a magic login link is sent to the user's email address, and the view shown in Figure 12 is rendered. Here, user-mfe replaces its form with a message that asks the user to check their inbox and to click the link to log in, while cart-mfe and product-mfe stay active. When the user clicks on the link, they are first taken to `/30724/login/verify`, where the login is verified, and are then redirected back to the cart view at `/30723/cart`.

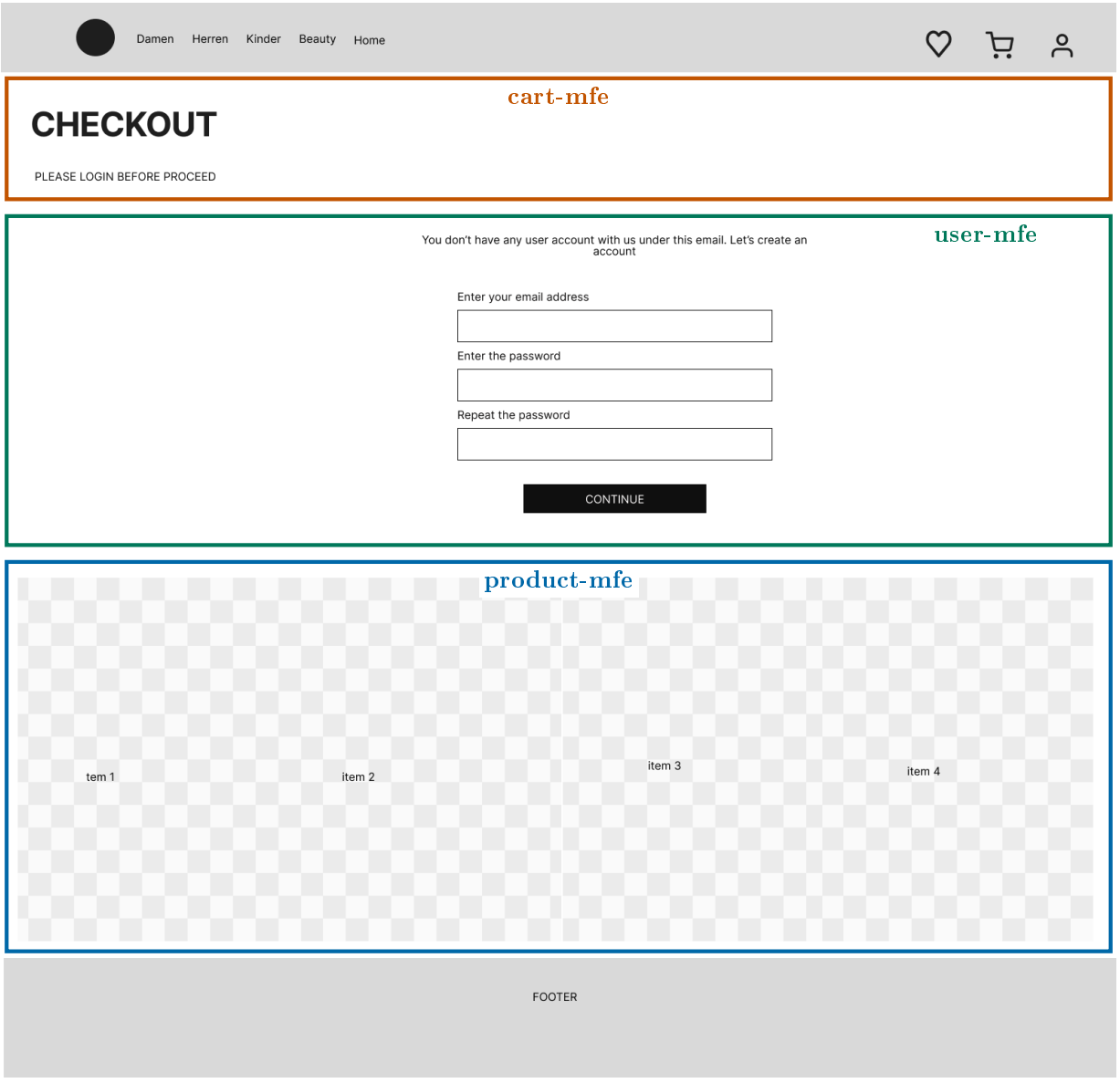


Figure 11: Checkout register view

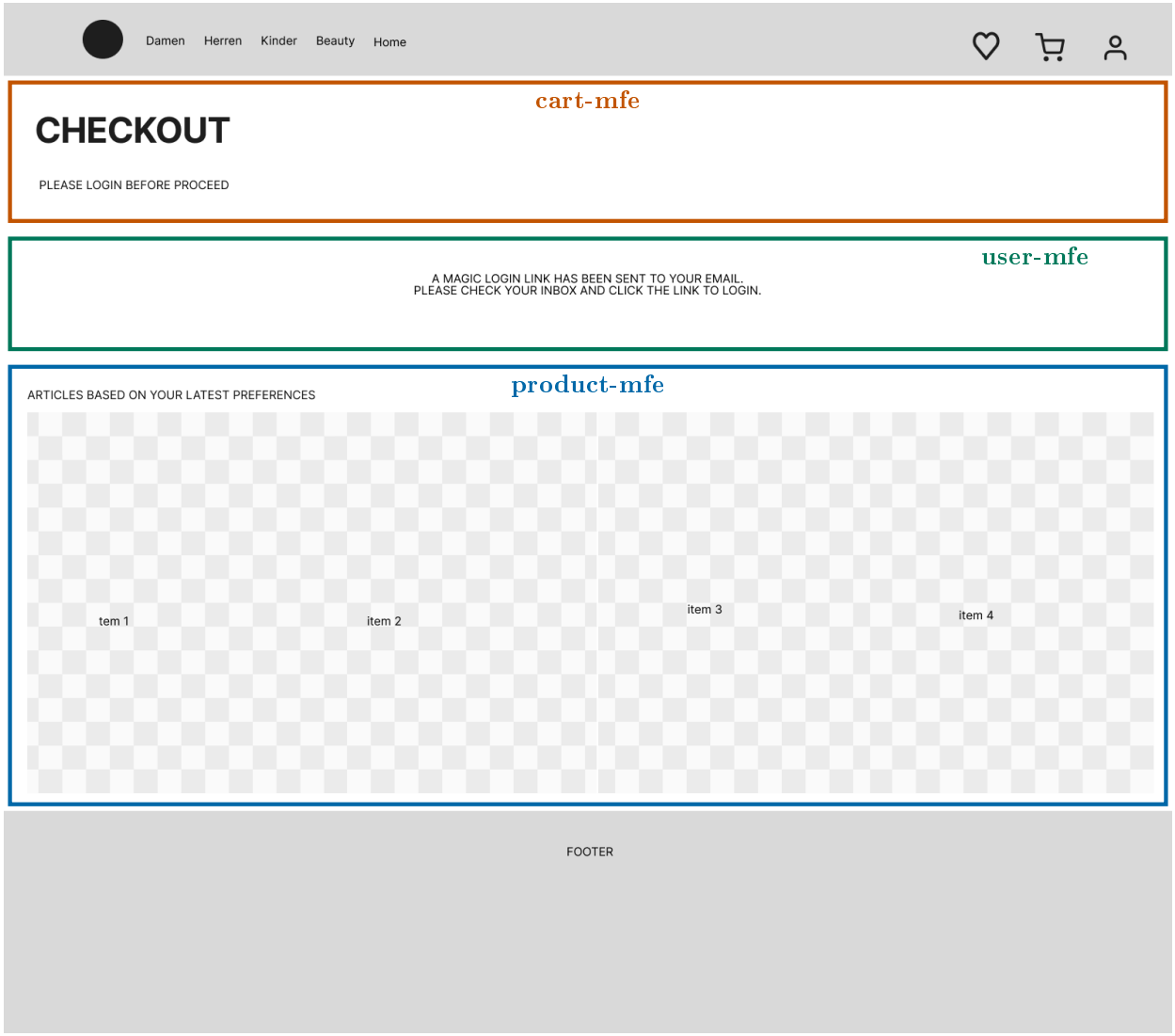


Figure 12: Checkout login confirmation view after a successful login or registration